Day 1: Introduction to Network Protocols

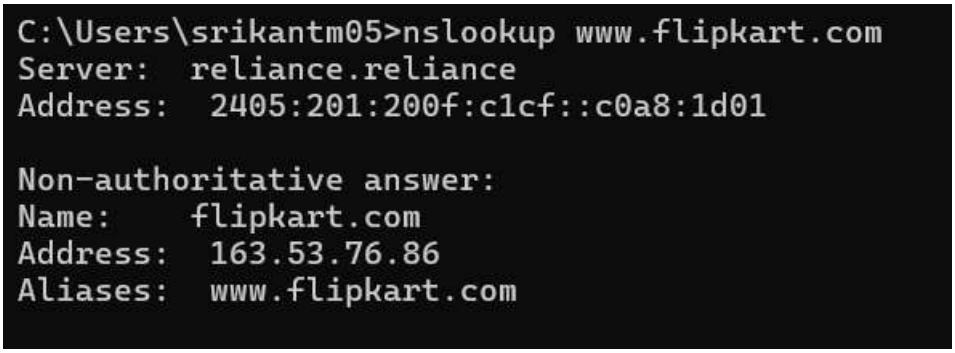
Task 1: Use Wireshark to capture network traffic while you load www.spotify.com in your browser, then identify and note down at least two DNS packets and two HTTP or HTTPS packets in the capture.

Two DNS packets were identified in the capture:

1. **DNS Query Packet**
    - Packet No: 40732
    - Type: Standard Query
    - Domain: moiazureorigin.clo.footprintdns.com
    - Description: The client requested the IP/record for the domain.
2. **DNS Response Packet**
    - Packet No: 40750
    - Type: Standard Query Response
    - Result: No such name
    - Description: The DNS server responded that the domain does not exist.

- **HTTP Packet 1:** Packet 4410 – GET request for `/rootr3/...`
- **HTTP Packet 2:** Packet 11649 – HTTP/1.1 200 OK response

Task 2: Simulate a DNS lookup for www.flipkart.com using the nslookup command in your terminal and write down the IP address returned along with a brief explanation of what this result means.



- `nslookup` = manually generates DNS traffic
- The IP address (163.53.76.86) is the final result
- Non-authoritative means the DNS server is not the original website server of the domain or it got the answer from cache or another DNS server

Task 3 : Explain the difference between HTTP and HTTPS by listing 3 key differences, and give a real-world example of an app or website you use that relies on HTTPS for secure communication.

3 Key Differences

- **Encryption:** HTTP transmits data in unencrypted plain text, allowing anyone eavesdropping on the network to view the contents. HTTPS encrypts all transmitted data using **SSL/TLS** (Secure Sockets Layer / Transport Layer Security) protocols.
- **Port Numbers:** HTTP communicates over network port **80** by default, whereas HTTPS operates over port **443**.
- **Authentication & Data Integrity:** HTTP provides no verification of server identity or data protection. HTTPS requires digital SSL/TLS certificates issued by a trusted Certificate Authority (CA) to verify the server's identity and ensure data isn't intercepted or modified in transit.

Real-World Example

Amazon relies on HTTPS to protect online shopping transactions. When you enter your login credentials, home address, or credit card details on `[https://www.amazon.com](https://www.amazon.com)`, HTTPS encrypts that data before sending it across the internet. This prevents hackers—especially on untrusted or public Wi-Fi networks—from intercepting your passwords or financial information.

Task 4: Open Wireshark and filter for DNS protocol traffic. Try searching for a new website (one you haven't visited recently) in your browser and describe the sequence of DNS queries and responses you observe.

Hint: Look for packets with 'Standard query' and 'Standard query response' in the Info column.

Step 1:

DNS Query (Client → DNS Server)

Example from your capture:

No: 19573
Protocol: DNS
Info: Standard query 0x2789 A www.bharatmatrimony.com

What it means:

- Your system (client) is asking:“What is the IP address of www.bharatmatrimony.com?”

Step 2: DNS Response (DNS Server → Client)

```
No: 19585
Protocol: DNS
Info: Standard query response 0x2789 A www.bharatmatrimony.com CNAME ...
```

What it means: DNS server replies with:

- A **CNAME record** (alias)
- The domain is redirected to another domain (Akamai CDN)

Step 3: Additional DNS Queries (for related domains)

From your capture: `Standard query AAAA 10164810.fls.doubleclick.net`

What it means:

- Browser is also resolving:
  - Ads/tracking domains (DoubleClick)
- AAAA = request for **IPv6 address**

Step 4: Corresponding DNS Response

`Standard query response AAAA 10164810.fls.doubleclick.net CNAME dart.l.doubleclick.net`

What it means:

- Another **CNAME redirection**
- Final domain belongs to **Google's ad network**

When filtering DNS traffic in Wireshark, I observed a sequence of DNS query and response packets. First, the client sent a **Standard query** for `www.bharatmatrimony.com` requesting its IPv4 address (A record). The DNS server responded with a **Standard query response**, which included a CNAME record redirecting the domain to an Akamai edge server.

Additional DNS queries were observed for related domains such as `doubleclick.net`, where the client requested IPv6 addresses (AAAA records). The DNS server responded with corresponding CNAME records pointing to Google ad servers.

This shows that when accessing a website, multiple DNS queries and responses occur, including resolution of the main domain and associated services like CDNs and advertisements.

Task 5: Pick any one protocol from DNS, DHCP, FTP, or SNMP (other than HTTP/HTTPS) and write a short scenario describing how it would be used in a real-world app or service you use (for example, how DHCP helps your phone connect to WiFi when you open Instagram at a cafe).

DNS (Domain Name System)

Real-World Scenario: Navigating to Uber Eats on Your Phone

When we use our phone browser and type `ubereats.com` to order food. Computers don't communicate using human-readable names like `ubereats.com`—they route traffic using numerical IP addresses (such as `13.225.89.41`). DNS acts as the internet's automated phonebook to bridge this gap in seconds:

- Querying the Resolver: When you hit enter, your phone sends a request to a DNS Resolver (usually provided by your internet provider or mobile carrier) asking, *"What is the numerical IP address for `ubereats.com`?"*

- Searching the Hierarchy: If the resolver doesn't have the answer saved, it queries the Root DNS Servers, then the .com TLD (Top-Level Domain) Servers, and finally the Authoritative Name Server for Uber.
- Returning the IP: Uber's authoritative server returns the exact IP address associated with `ubereats.com` back to your resolver, which then hands it directly to your browser.

The phone immediately uses that IP address to establish a secure connection to Uber's servers, loading the homepage and restaurant options seamlessly without you ever having to remember a string of numbers.